NATURAL LANGUAGE PROCESSING · COMPANION READER

Neural, LLM & Agentic Approaches to POS Tagging

From per-word guessing to Bi-LSTM-CRFs, Transformers, prompting, and tool-using agents.

Session 7 (Dr. Chetana Gavankar) · Read alongside the lecture slides · Every slide example worked in full

How to use this companion. The slides give you the architectures and the headline numbers. This reader gives you the *why*. Every idea is explained in plain words. Every formula is built up one symbol at a time. Every example is worked with real numbers, with no step skipped. Five colour-coded boxes guide you:

- The **blue** "intuition" box gives the core idea in one breath.
- The **amber** "everyday picture" box ties it to real life, before any math.
- The **green** "worked example" box solves a problem step by step.
- The **red** "watch out" box flags the common mistake.
- The **purple** "key takeaway" box is the one line to remember.

Read it alongside the slides, in order. Its sibling file, `lecture.html`, lets you *play* with each idea live.

1 What this session covers

Part-of-speech (POS) tagging asks one small question for every word in a sentence. Is this word a noun, a verb, an adjective? Earlier sessions answered it with hand-built probability models: HMMs, MEMMs, and classic CRFs. This session tells the modern story. We will see how **deep learning** pushed accuracy from about 96% to near a human's. We will also see how tagging became one small job inside a large language model.

We climb a single ladder. Each rung leans on the one below it:

1. **The neural baseline — Bi-LSTM-CRF.** Turn words into vectors (embeddings). Read the sentence both ways with an LSTM. Then let a CRF pick the best *whole-sentence* tag sequence. (Slides 4–6.)
2. **The Transformer.** Drop the slow left-to-right reading. Let every word look at every other word at once, through **self-attention**. (Slide 7.)
3. **Pre-trained models — BERT, RoBERTa.** Start from a model that already "knows" language. Add a one-line classifier on top. (Slides 8–9.)
4. **Prompting an LLM.** Skip training. Just *ask* a large language model to tag the sentence.

(Slide 10.)

5. **LLM agents.** Let an LLM *call* a fast tagger as a tool inside a bigger task. (Slide 11.)

☞ The intuition

The session has one question all the way through. How much of the sentence can a model use, and how cheaply? Each method reads more of the sentence than the last. We start with a single word. Then its neighbours. Then every word at once. Finally, all the language an LLM ever read.

2 Why tagging is hard: one word, many jobs

If every word had just one tag, a dictionary would finish the job. The trouble is that the same word can change its part of speech. "Book" is a noun in "read a book." It is a verb in "book a flight." So the tagger must *choose*. It reads the nearby words to decide the role this time. The technical name for this is **disambiguation** — picking the right meaning.

★ Everyday picture

Think of a word as a person with several possible jobs. "Coach" could be a bus, a trainer, or the act of training. You do not know which until you see the company it keeps. "The team's new *coach*" is a trainer. "I will *coach* you" is an action. Context is the name-tag at the party.

We write the task like this. Given words $w_1, w_2, \ldots, w_n$, choose one tag per word from a fixed set. This course uses the **Penn Treebank** tag set. The tags you will meet here are:

- **NN** singular noun, **NNS** plural noun.
- **VB** base verb, **VBZ** present verb ending in -s, **VBD** past verb.
- **DT** determiner ("the," "a"), **JJ** adjective, **IN** preposition, **PRP** pronoun.

A first, simple tagger gives each word its *most frequent* tag from training text. This is the **unigram baseline**. It is a useful yardstick. It also shows exactly why we need context.

✎ Worked example: the unigram baseline trips on "She books a flight"

Tag **"She books a flight ."** with a tagger that always picks each word's single most common tag.

Step 1. **Tag the easy words.** "She" is almost always **PRP**. "a" is **DT**. "flight" is **NN**. "." is **.**. Four words, four safe tags.

Step 2. **Hit the hard word.** The word "books" is a plural noun (**NNS**, "the books") more often than a verb (**VBZ**, "she books"). Say it is **NNS** 70% of the time. So the tagger guesses **NNS**.

Step 3. **Score it.** The true tag here is **VBZ**. "She *books* a flight" is an action. So the tagger gets 5 of 6 tokens right. That is **83%** on this sentence, and it missed the one word that mattered.

Step 4. **See the fix.** The word before is "She." The words after are "a flight." A pronoun, then a verb, then "a + noun" is the natural pattern. Any model that

reads those neighbours flips the answer to **VBZ**.

So the rest of this session is about models that read the neighbours. They read ever more of them, ever more directly.

✗ Watch out

"97% accuracy" hides where the errors live. They cluster on the ambiguous, high-information words ("books," "that," "back"). Those are the words a later system most needs right. Judge a tagger by its hard cases, not its easy ones.

Where this shows up in ML. The unigram baseline is a **lookup table**. It is the simplest possible model, with zero context. Every method that follows adds context in a smarter way. Keeping this baseline in mind makes the gains concrete. Each rung of the ladder erases some of those context-free errors.

✓ Key takeaway

Tagging is not lookup. It is **disambiguation in context**. The hard words have several possible tags, and only the nearby words reveal which one is right.

3 Step 1 — Words into vectors: word & character embeddings

A neural network cannot read letters. It reads numbers. So the first job is to turn each word into a vector — a short list of numbers — that captures its meaning. A good encoding places similar words close together. Then the model can carry what it learns from "dog" over to "cat," without seeing every word in every context.

☞ The intuition

An **embedding** is a word's address in "meaning space." Words used the same way get nearby addresses. The model never sees the letters `c-a-t`. It sees a point that sits near `dog` and far from `Tuesday`.

★ Everyday picture

Picture a big map where towns are words. Towns that trade with the same partners end up near each other, even if you never compared them directly. "King" and "queen" are neighbouring towns. "King" and "cat" are on different continents. The map's coordinates *are* the embedding.

Two kinds of embedding feed the tagger:

- **Word embeddings** (**Word2Vec**, **GloVe**): one fixed vector per known word, learned from how words sit together in huge text.
- **Character embeddings**: a small network reads a word *letter by letter*. This rescues **out-of-vocabulary** words — words never seen in training. The spelling itself carries clues. The suffix "-ly" hints at an adverb; "-ing" hints at a verb.

We measure how close two vectors are with the **cosine similarity**. It is the cosine of the angle

between them. It is +1 for the same direction, 0 for unrelated, and negative for opposite:

$$\cos(\mathbf{a},\mathbf{b}) \;=\; \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|}, \qquad \mathbf{a}\cdot\mathbf{b}=\sum_i a_i b_i, \quad \|\mathbf{a}\|=\sqrt{\textstyle\sum_i a_i^2}. \tag{1}$$

Here $\mathbf{a}\cdot\mathbf{b}$ is the **dot product**: multiply matching entries, then add them up. And $\|\mathbf{a}\|$ is the vector's length.

✎ Worked example: "king" sits near "queen," far from "cat"

Use tiny 2-D embeddings. king $=[0.9, 0.8]$, queen $=[0.85, 0.9]$, cat $=[-0.7, 0.6]$.

Step 1. **Lengths.** $\|\text{king}\| = \sqrt{0.9^2+0.8^2} = \sqrt{1.45} = 1.204$. $\|\text{queen}\| = \sqrt{0.85^2+0.9^2} = \sqrt{1.5325} = 1.238$. $\|\text{cat}\| = \sqrt{0.49+0.36} = \sqrt{0.85} = 0.922$.

Step 2. **king vs queen.** Dot product $0.9(0.85)+0.8(0.9) = 0.765+0.72 = 1.485$. So $\cos = \frac{1.485}{1.204\times1.238} = \frac{1.485}{1.490} = \mathbf{0.996}$. That is almost 1: nearly the same direction.

Step 3. **king vs cat.** Dot product $0.9(-0.7)+0.8(0.6) = -0.63+0.48 = -0.15$. So $\cos = \frac{-0.15}{1.204\times0.922} = \frac{-0.15}{1.110} = \mathbf{-0.135}$. That is near 0: basically unrelated.

Step 4. **Read it.** The geometry holds grammar and meaning. "King" and "queen" behave alike. So a tagger that learns about one transfers to the other.

For an unknown word, the character network still builds a vector from the spelling. We **concatenate** (glue end to end) the word vector and the character vector into one input. If the word part is 4 numbers and the character part is 2, the input for that token is a single 6-number vector:

$$\mathbf{x} \;=\; [\;\underbrace{0.9,\ 0.8,\ -0.2,\ 0.1}_{\text{word part}}\;|\;\underbrace{0.7,\ -0.3}_{\text{char part}}\;]. \tag{2}$$

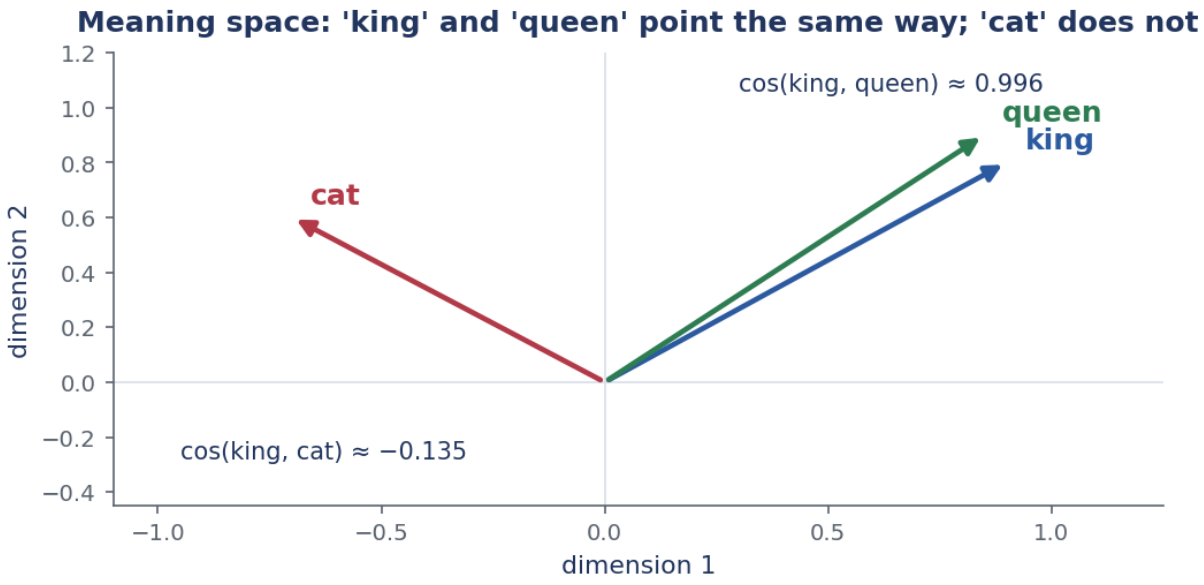

Figure 1: Tiny 2-D meaning space. "King" and "queen" point almost the same way (cosine ≈ 0.996). "Cat" points off on its own (cosine ≈ -0.135). Similar words get similar vectors, so learning transfers.

✗ Watch out

Plain Word2Vec and GloVe give one vector per word, *whatever the context*. So "bank" gets the same vector by the river and in finance. That is fine as an *input* here, because the Bi-LSTM on top adds the context. The contextual fix comes later, with BERT (Section 8).

Where this shows up in ML. Embeddings are the first layer of almost every modern NLP model. The same trick shows up everywhere. Learn a vector per token, so similar tokens land nearby. It powers search, recommendation, and the input layer of every LLM.

✓ Key takeaway

Turn each word into a vector, so similar words sit close together. Add a character-level vector, so an unknown word still gets a sensible encoding from its spelling.

4 Step 2 — The Bi-LSTM: reading the sentence both ways

Now the tagger reads the word-vectors in order. For each position it builds a summary of everything around it. The workhorse is the **LSTM** (Long Short-Term Memory). It is a recurrent network with a memory that it learns to update word by word.

A plain **recurrent network** (RNN) keeps a running summary h_t. It updates the summary at each word, from the last summary and the current input. This works, but over a long sentence the early words fade. This is the **vanishing gradient** problem: the learning signal shrinks toward zero as it travels back through many steps.

☞ The intuition

An LSTM is a notepad with three dials. As each word arrives, it decides three things. How much of the old notes to *keep*. How much of the new word to *write down*. How much of the notepad to *read out* right now. Because it can keep a note unchanged, important facts survive across a long sentence.

★ Everyday picture

Imagine taking minutes in a long meeting. You do not rewrite the page every sentence. You keep the decisions that still matter (high "keep"). You jot the genuinely new points (high "write"). When someone asks, you read out only the relevant lines (the "read" dial). The LSTM's three gates are these three habits.

The three **gates** are sigmoids. Each gives a number in $(0, 1)$ — a dial from "closed" to "open." The sigmoid is $\sigma(z) = \frac{1}{1+e^{-z}}$. Write x_t for the current word-vector and h_{t-1} for the previous summary:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad \text{forget: keep how much old memory?} \tag{3}$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad \text{input: write how much new?} \tag{4}$$

$$g_t = \tanh(W_g x_t + U_g h_{t-1} + b_g) \quad \text{candidate: the new content itself} \tag{5}$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad \text{output: reveal how much?} \tag{6}$$

The W's and U's are learned weight matrices. The b's are biases. The **cell state** c_t (the notepad) and the **hidden state** h_t (what we read out) update as

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t), \tag{7}$$

where $\odot$ means multiply entry by entry. Read c_t aloud: "keep f_t of the old memory, then add i_t of the new content g_t."

✎ Worked example: one LSTM step, every number shown

Use single-number (scalar) states, so the arithmetic is easy to see. Start with old memory $c_{t-1} = 0.5$, old summary $h_{t-1} = 0.3$, and new word $x_t = 1.0$. The learned weights for this cell are given in each step.

Step 1. Forget gate. With W_f=0.5, U_f=0.4, b_f=0.1: $z = 0.5(1.0) + 0.4(0.3) + 0.1 = 0.72$, so $f_t = \sigma(0.72) = \mathbf{0.673}$. Keep about two-thirds of the old memory.

Step 2. Input gate. With W_i=0.6, U_i=0.3, b_i=0: $z = 0.69$, so $i_t = \sigma(0.69) = \mathbf{0.666}$.

Step 3. Candidate. With W_g=0.7, U_g=0.2, b_g=0: $z = 0.76$, so $g_t = \tanh(0.76) = \mathbf{0.641}$. This is the proposed new note.

Step 4. Update the memory. $c_t = f_t\, c_{t-1} + i_t\, g_t = 0.673(0.5) + 0.666(0.641) = 0.336 + 0.427 = \mathbf{0.763}$.

Step 5. Output gate. With W_o=0.4, U_o=0.5, b_o=0.1: $z = 0.65$, so $o_t = \sigma(0.65) = \mathbf{0.657}$.

Step 6. Read out. $h_t = o_t \tanh(c_t) = 0.657 \times \tanh(0.763) = 0.657 \times 0.643 = \mathbf{0.422}$.

So this word's summary is $h_t \approx 0.422$. The cell carried two-thirds of its old memory forward and folded in new information. No rewrite, no forgetting everything.

The "Bi" in **Bi-LSTM** means we run *two* LSTMs. One reads left to right (forward, $\overrightarrow{h_t}$). One reads right to left (backward, $\overleftarrow{h_t}$). We glue their summaries together, $h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}]$. So each word's representation sees *both* sides at once.

★ Everyday picture

Take "bank" in "I walked to the river bank ." The forward LSTM has already read "…river," so it leans toward riverbank, a noun. The backward LSTM has read the final "." coming the other way. Only together do they pin "bank" down. One-way reading would miss half the evidence.

✗ Watch out

The gates are *learned*, not set by hand. Training discovers when to keep and when to forget. And even an LSTM strains over very long distances. That limit is exactly what self-attention removes (Section 6).

Where this shows up in ML. Before Transformers, LSTMs powered machine translation, speech recognition, and text generation. The core idea is a learned gate that decides how much information flows. It is everywhere, from GRUs to the gating inside modern models.

✓ Key takeaway

A Bi-LSTM gives every word a context-aware vector. It reads the sentence both ways and uses gated memory to carry information across long distances.

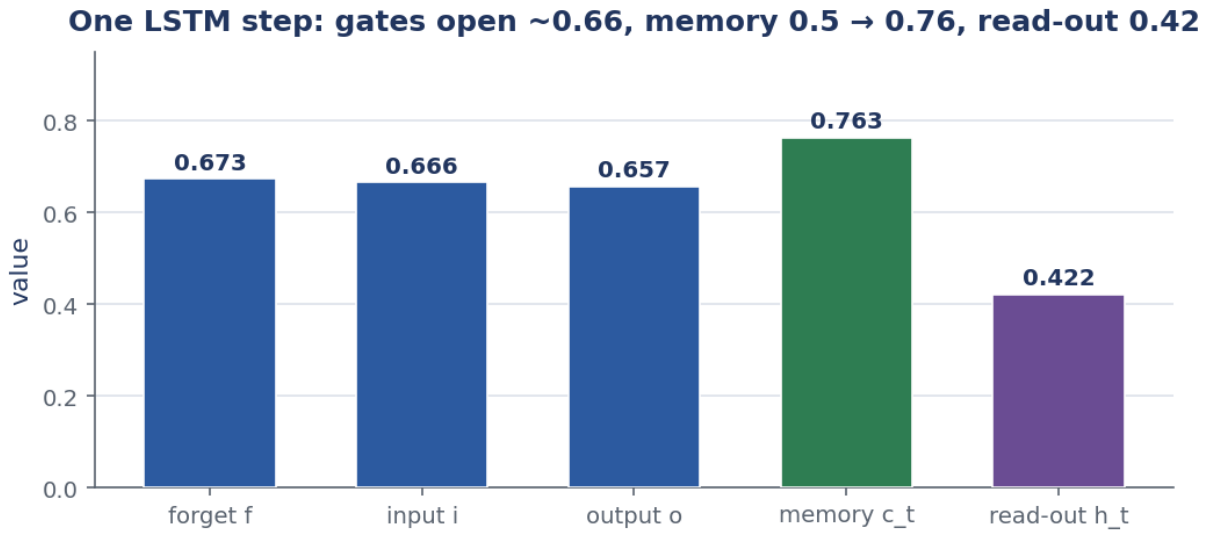

Figure 2: The worked LSTM step. The three gates open to f=0.67, i=0.67, o=0.66. The memory moves from c_{t-1}=0.5 to c_t=0.76, and the read-out is h_t=0.42. The forget gate staying open is what lets early words survive to the end.

5 Step 3 — The CRF: choosing the best *whole* sequence

The Bi-LSTM scores each word's tags on its own. But tags are not independent. A determiner is almost never followed by a verb. The **CRF** (Conditional Random Field) layer fixes this. It scores the *entire* tag sequence at once and picks the best one. So the answer is always grammatically sensible.

☞ The intuition

Tagging each word alone is like every musician playing their loudest note alone. The result can be noise. The CRF is the conductor. It picks the set of notes that sound best *together*, even if one player must soften a personal favourite.

★ Everyday picture

Filling a crossword, you do not commit each square to its single likeliest letter. You choose letters that make every crossing word legal. The CRF does this for tags. It trades a slightly less likely tag on one word for a sequence that fits the grammar overall.

The CRF uses two kinds of score. The **emission** score $E_i[t]$ comes from the Bi-LSTM: how well tag t fits word i on its own. The **transition** score $T[a \to b]$ is a small learned table: how natural it is for tag b to follow tag a. A whole sequence's score adds them all up:

$$\text{Score}(t_1, \ldots, t_n) \;=\; \sum_{i=1}^{n} E_i[t_i] \;+\; \sum_{i=2}^{n} T[t_{i-1} \to t_i]. \tag{8}$$

To turn scores into probabilities, we do two things. First, make every score positive with exp. Then divide by the total over *every* possible sequence. That total is the **partition function** Z:

$$P(t_1, \ldots, t_n) \;=\; \frac{\exp\big(\text{Score}(t_1, \ldots, t_n)\big)}{Z}, \qquad Z = \sum_{\text{all tag sequences}} \exp\big(\text{Score}(\cdot)\big). \tag{9}$$

Training pushes up the true path's score relative to Z. At test time the **Viterbi algorithm** finds the single best-scoring sequence quickly, without listing them all.

✎ Worked example: "the plays" — greedy picks a verb, the CRF picks the noun

Two words, three candidate tags {DT, NN, VBZ}. The Bi-LSTM **emission** scores E and the learned **transition** table T are:

Emission $E_i[t]$

	DT	NN	VBZ
the	5	0	−2
plays	−3	2	3

Transition T[row → col]

from \ to	DT	NN	VBZ
DT	−2	**4**	−1
NN	−1	1	3
VBZ	2	2	−3

Step 1. What greedy does. Per word, take the biggest emission. "the" → DT (5). "plays" → VBZ (3). Greedy answer: DT VBZ.

Step 2. Score every full path as $E_1[t_1] + T[t_1 \to t_2] + E_2[t_2]$:

$t_1 \backslash t_2$	DT	NN	VBZ
DT	5−2−3 = 0	5+4+2 = **11**	5−1+3 = 7
NN	0−1−3 = −4	0+1+2 = 3	0+3+3 = 6
VBZ	−2+2−3 = −3	−2+2+2 = 2	−2−3+3 = −2

Step 3. Find the best. The maximum is **11** at DT NN. That beats the greedy DT VBZ (score 7). The reward T[DT → NN] = +4 outweighs the higher emission for VBZ.

Step 4. Turn into a probability. Sum exp(score) over all nine paths: $e^{11}+e^{7}+e^{6}+e^{3}+e^{2}+e^{0}+e^{-2}+e^{-3}+e^{-4} \approx 61402.9 = Z$. Then

$$P(\text{DT NN}) = \frac{e^{11}}{Z} = \frac{59874.1}{61402.9} = \mathbf{0.975}.$$

Step 5. Read it. The CRF puts **97.5%** on "the plays" as determiner-plus-noun. It puts only 1.8% on the greedy verb reading. Structure won.

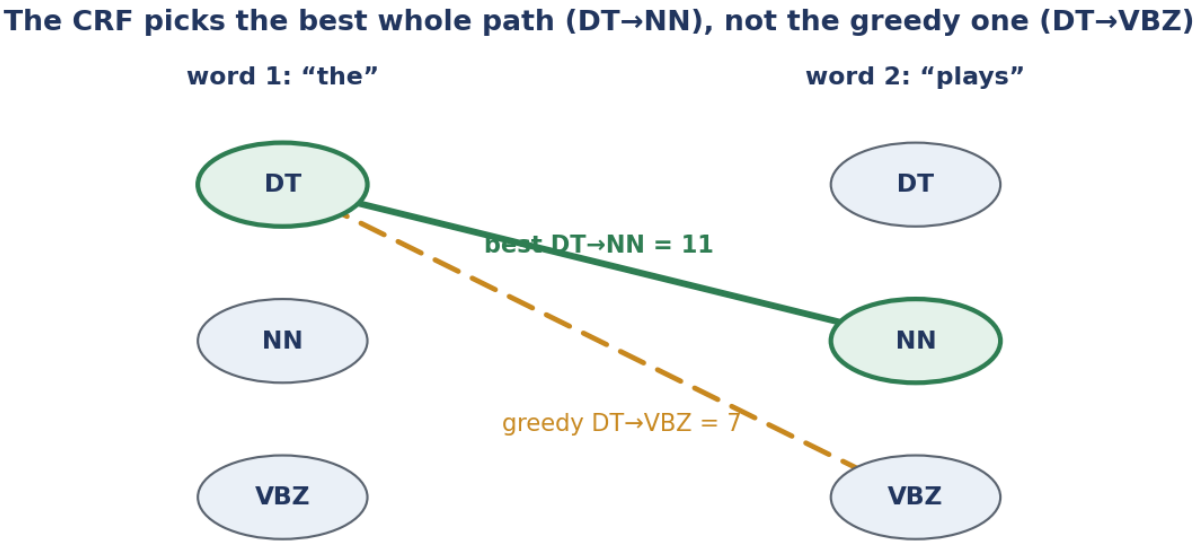

Figure 3: The two-word lattice. Each column is a word; each dot is a candidate tag. The greedy per-word choice (DT VBZ, dashed) scores 7. The CRF's best path (DT NN, solid) scores 11, because the DT→NN transition is strongly rewarded.

✗ Watch out

Do not confuse the two scores. **Emission** is word-fits-tag, from the Bi-LSTM. **Transition** is tag-follows-tag, from the CRF table. Greedy, per-word decoding throws away the transition scores. That is exactly the information that keeps a sequence legal.

Where this shows up in ML. A CRF layer sits on top of sequence models for named-entity recognition, chunking, and segmentation. It fits anywhere the output is a *structured* sequence with rules. It is the classic example of **structured prediction**: score the whole object, not each part alone.

✓ Key takeaway

The CRF adds learned tag-to-tag transition scores. It decodes the single best-scoring *sequence*, so the tagging is globally consistent, not locally greedy.

6 How well Bi-LSTM-CRF works — and why 1% still matters

Put the three steps together — embeddings, Bi-LSTM, CRF — and you get the **Bi-LSTM-CRF**. It is the standard neural baseline for sequence tagging. On the Penn Treebank benchmark it reaches about 97%–98% **token accuracy**. All features are learned, with no hand-written rules.

☞ The intuition

"97% per word" sounds finished. But a sentence is a chain. It is fully correct only if *every* word is correct. Small per-word errors multiply into a much higher chance that *some* word in a long sentence is wrong.

If each word is right with probability a, and errors are independent, a sentence of n words is fully correct with probability a^n. This drops fast as n grows.

✎ Worked example: how often is a 20-word sentence perfect?

Compare a Bi-LSTM-CRF at $a = 0.97$ with a Transformer at $a = 0.99$, on $n = 20$ words.

Step 1. **Bi-LSTM-CRF.** 0.97^{20}. Since $\ln 0.97 = -0.03046$ and $20 \times (-0.03046) = -0.6092$, we get $0.97^{20} = e^{-0.6092} = \mathbf{0.544}$. Only about **54%** of 20-word sentences come out perfect.

Step 2. **Transformer.** 0.99^{20}. Here $\ln 0.99 = -0.01005$ and $20 \times (-0.01005) = -0.2010$, so $0.99^{20} = e^{-0.2010} = \mathbf{0.818}$. That is about **82%** perfect.

Step 3. **So what.** Going from 97% to 99% per word looks like a tiny 2 points. But it lifts *whole-sentence* correctness from 54% to 82%. That is why the last percent is worth chasing.

Pros and cons. The Bi-LSTM-CRF is very accurate. With transfer learning, it works well even for **low-resource** languages, which have little training data. Its drawbacks: it reads words one at a time, so training is slow. And it was overtaken by the Transformer, our next step.

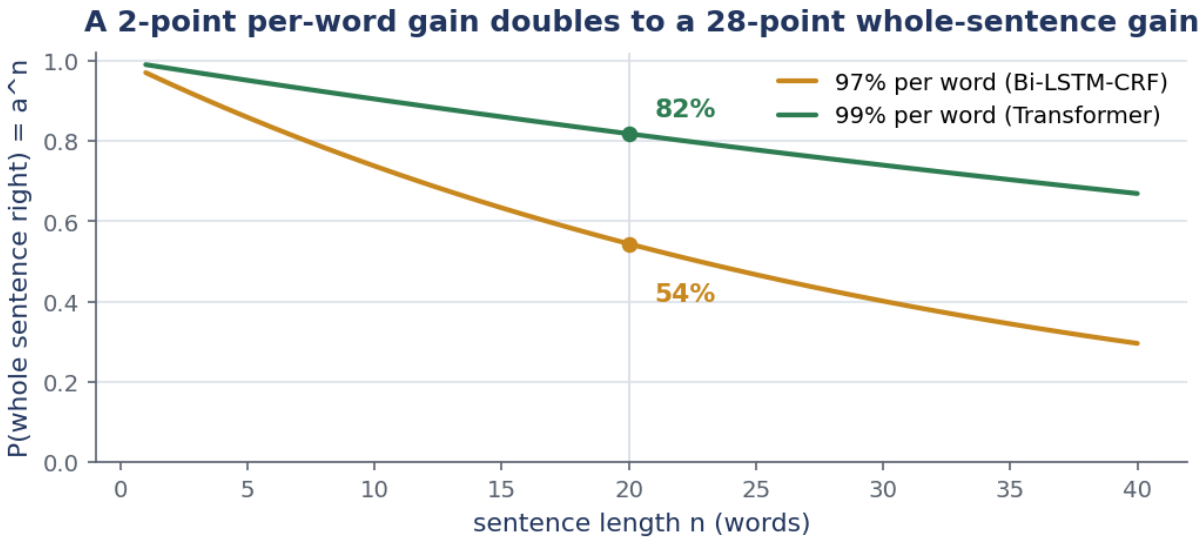

Figure 4: Whole-sentence accuracy a^n against sentence length n. At 97% per token (amber) it sinks below 60% by $n = 20$. At 99% (green) it stays much higher. A small per-word gain compounds over a sentence.

✗ Watch out

Real errors are not perfectly independent, so a^n is an estimate, not a law. But the lesson holds. Per-token accuracy flatters a tagger. Always check sentence-level accuracy for tasks that need the whole sentence right.

Where this shows up in ML. The token-versus-sequence accuracy gap appears wherever outputs are structured: translation, code generation, speech. It is why leaderboards report exact-match alongside per-token scores.

✓ Key takeaway

Bi-LSTM-CRF hits about 97–98% per token on the Penn Treebank. Because errors compound across a sentence, even a one-point per-word gain is a large whole-sentence gain. That motivates everything that follows.

7 Step 4 — The Transformer: every word sees every word at once

The LSTM's weakness is that it reads in order, one word at a time. So distant words talk only through a long chain. The **Transformer** throws out the chain. Its engine, **self-attention**, lets every word gather information directly from every other word in a single step. Near or far, the cost is the same.

☞ The intuition

Self-attention is a meeting. To understand your own role, you ask everyone else "how relevant are you to me?" You listen *more* to the relevant people. You blend what they say into your own notes. Distance around the table does not matter; relevance does.

★ Everyday picture

To tag "bank" in "the river bank ," you glance at "river" and barely at "the." Self-attention makes that glance a computation. Score every word for relevance, then pull in mostly from the ones that score high.

Each word becomes three vectors, through learned matrices. A **query** q (what am I looking for?), a **key** k (what do I offer?), and a **value** v (the information I carry). For a query word, score every word by the dot product $q \cdot k_j$. Scale by $1/\sqrt{d}$ to keep the numbers tame (d is the vector size). Turn the scores into weights with softmax, then mix the values:

$$\alpha_j \;=\; \mathrm{softmax}_j\!\left(\frac{q \cdot k_j}{\sqrt{d}}\right), \qquad \text{output} \;=\; \sum_j \alpha_j \, v_j. \tag{10}$$

The weights α_j are positive and add to 1. They are a recipe for how much of each word to blend in. Stack several such "heads" in parallel (**multi-head attention**), add a small feed-forward network, and repeat in layers. That is a Transformer encoder.

✎ Worked example: tagging "bank": attention locks onto "river"

Query the word "bank" against three context words. Use 2-D vectors and $d = 2$, so $\sqrt{d} = 1.414$. Let $q_{\text{bank}} = [1, 2]$ and the keys $k_{\text{river}} = [1, 2]$, $k_{\text{the}} = [2, 0]$, $k_{\cdot} = [0, 1]$.

Step 1. **Raw scores (dot products).** $q \cdot k_{\text{river}} = 1(1)+2(2) = 5$. $q \cdot k_{\text{the}} = 1(2)+2(0) = 2$. $q \cdot k_{\cdot} = 1(0) + 2(1) = 2$.

Step 2. **Scale by $1/\sqrt{2}$.** $5/1.414 = 3.54$. $2/1.414 = 1.41$. $2/1.414 = 1.41$.

Step 3. **Softmax to weights.** $e^{3.54} = 34.3$, $e^{1.41} = 4.11$, $e^{1.41} = 4.11$; sum $= 42.6$. So $\alpha_{\text{river}} = 34.3/42.6 = \mathbf{0.807}$ and $\alpha_{\text{the}} = \alpha_{\cdot} = 4.11/42.6 = \mathbf{0.097}$.

Step 4. **Blend the values.** With value vectors $v_{\text{river}} = [0, 1]$ (a noun signal), $v_{\text{the}} = [1, 0]$, $v_{\cdot} = [0.5, 0.5]$:

$$\text{output} = 0.807[0, 1] + 0.097[1, 0] + 0.097[0.5, 0.5] = [\,0.145,\ \mathbf{0.855}\,].$$

Step 5. **Read it.** "Bank" pulled **81**% of its information straight from "river." That gives a strong noun signal (0.855 on the noun dimension). And "river" could have been 50 words away. The cost would be identical.

✗ Watch out

Comparing every word with every word costs $\mathcal{O}(n^2)$ work. That is fine for a sentence, heavy for a whole book. And an attention weight tells you what the model *looked at*, not the final tag. The tag comes from the layers built on top.

Where this shows up in ML. Self-attention *is* the Transformer. The Transformer is the backbone of BERT, GPT, and every modern LLM. The exact move here repeats inside them. Score with $q \cdot k$, normalise with softmax, then blend the values. That is how an LLM decides which earlier words matter for the next one.

✓ Key takeaway

Self-attention lets each word gather information from every other word in one parallel step, weighting by relevance. So long-range context is direct, and distance no longer fades the

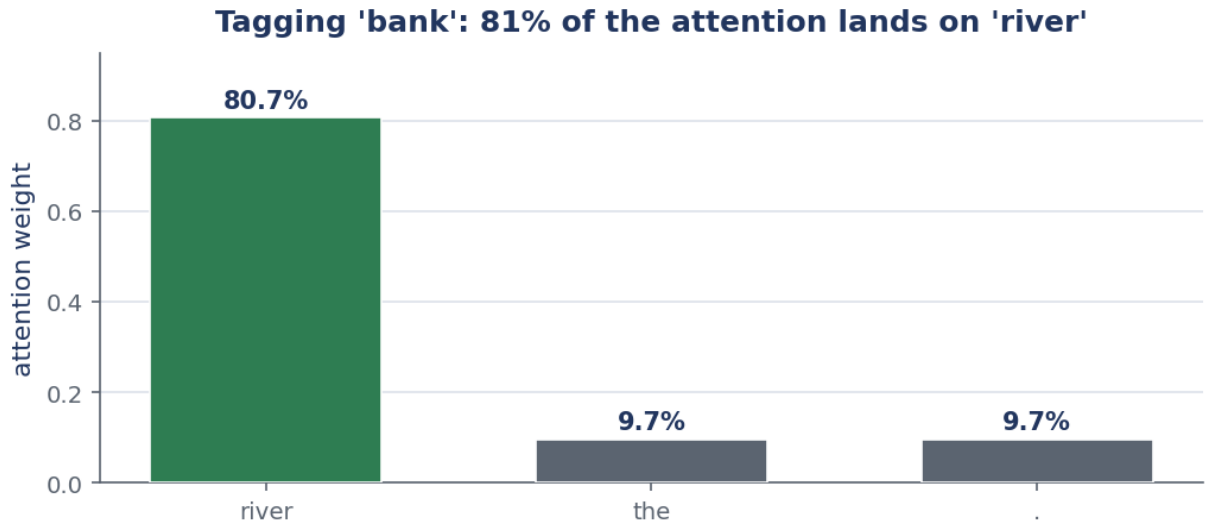

Figure 5: Attention weights when tagging "bank." Of all the context, 81% of the weight lands on "river" and only about 10% each on "the" and ".". The model reaches the relevant word directly, with no chain to fade through.

signal.

8 Step 5 — Pre-trained models: fine-tuning BERT & RoBERTa

Why teach a model English from scratch for every task? **BERT** and **RoBERTa** are Transformers already trained on huge text. They arrive knowing grammar and a lot about the world. For POS tagging we just adapt them. This is **fine-tuning**, and it is far cheaper than starting over.

☞ The intuition

Pre-training is a general education. Fine-tuning is a short job induction. The model already understands sentences. We only teach it our specific labels and our label *names*.

★ Everyday picture

Hiring a well-read graduate, you do not reteach them English. You give them a one-day course on your house style — which tag goes on which word — and they are productive. BERT is that graduate. The POS corpus is the one-day course.

Pre-training uses the **masked language model** (MLM) trick. Hide a word and make the model predict it from both sides ("the river ___" → "bank"). To do this well, it must learn syntax and meaning. Fine-tuning then adds one **classification head** — a single linear layer — on top of each token's contextual vector $\mathbf{h}$. The head turns the vector into one **logit** (a raw score) per tag. Softmax turns logits into probabilities:

$$\text{logits} = W\mathbf{h} + \mathbf{b}, \qquad P(t) = \frac{e^{\text{logit}_t}}{\sum_{t'} e^{\text{logit}_{t'}}}. \tag{11}$$

Here W has one row of weights per tag. We then train the whole model for a few epochs on the tagging corpus.

✎ Worked example: reading the tag off "plays" in "she plays well"

Suppose BERT's contextual vector for "plays" is $\mathbf{h} = [0.5, -1.0, 2.0, 0.3]$. The head has three tag rows: $W_{\text{DT}} = [0.1, 0.2, -0.5, 0.0]$, $W_{\text{NN}} = [0.3, 0.1, 0.4, -0.2]$, $W_{\text{VBZ}} = [-0.2, -0.3, 0.9, 0.5]$ (biases 0).

Step 1. **Logit for DT.** $0.1(0.5)+0.2(-1)+(-0.5)(2)+0(0.3) = 0.05-0.2-1.0+0 =$ **-1.15**.

Step 2. **Logit for NN.** $0.3(0.5)+0.1(-1)+0.4(2)+(-0.2)(0.3) = 0.15-0.1+0.8-0.06 =$ **0.79**.

Step 3. **Logit for VBZ.** $-0.2(0.5)+(-0.3)(-1)+0.9(2)+0.5(0.3) = -0.1+0.3+1.8+0.15 =$ **2.15**.

Step 4. **Softmax.** $e^{-1.15} = 0.317$, $e^{0.79} = 2.203$, $e^{2.15} = 8.585$; sum $= 11.105$. So $P(\text{DT}) =$ **0.029**, $P(\text{NN}) =$ **0.198**, $P(\text{VBZ}) =$ **0.773**.

Step 5. **Read it.** The head reads off VBZ with 77% confidence. That is correct for "she *plays* well." The pre-trained vector already encoded "verb after she." The one-line head only had to point at it.

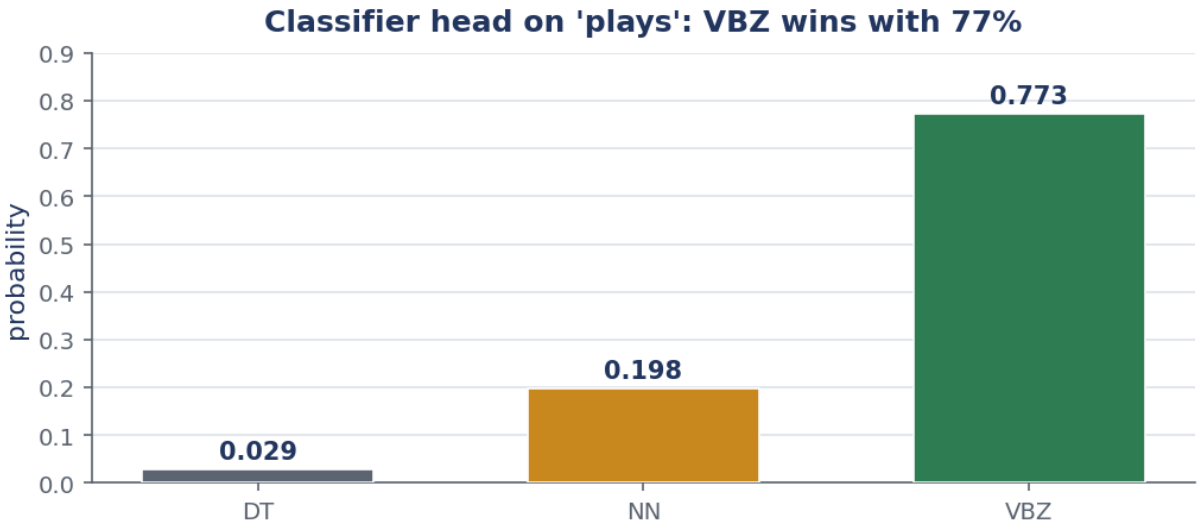

Figure 6: The classification head's output for "plays." Three logits (−1.15, 0.79, 2.15) become probabilities (0.03, 0.20, 0.77) through softmax. The tall green bar, VBZ, is the chosen tag.

✗ Watch out

BERT splits rare words into **sub-word** pieces ("playing" → "play" + "##ing"). The convention is to read the tag from the *first* piece. Forget this and words and tags drift apart, which quietly wrecks accuracy.

Where this shows up in ML. "Pre-train once, fine-tune cheaply" is the dominant recipe in modern NLP. The same pattern — a small head on a strong backbone — powers named-entity recognition, sentiment, and question answering.

✓ Key takeaway

Fine-tuning bolts a one-layer classifier onto a model that already understands language. So each token's contextual vector maps to a tag with very little extra training.

9 Why Transformers won: speed, depth, ecosystem

Fine-tuned Transformers are today's state of the art for POS tagging, at about 98.5%–99.5% **accuracy**. Three advantages explain the takeover. They are fast to train. They capture the deepest context. And they ship inside the tools everyone already uses.

☞ The intuition

An LSTM reads a sentence the way you read aloud — one word after another. It cannot start word t before finishing word $t-1$. A Transformer takes in the whole sentence at once, like photographing a page instead of reading it letter by letter. Same answer, far less waiting.

✎ Worked example: counting steps and counting mistakes

Step 1. **Sequential steps.** For a 50-word sentence the RNN needs 50 steps in order. Each step waits for the one before. Self-attention computes all positions together — about 1 parallel step per layer. With enough hardware, that is a 50× gap in what must happen in sequence.

Step 2. **Mistakes on a big corpus.** At 97% accuracy, a 1,000,000-token corpus has $0.03 \times 10^6 = \mathbf{30{,}000}$ wrong tags. At 99% it has $0.01 \times 10^6 = \mathbf{10{,}000}$. That is 20,000 **fewer errors**, a 3× cut, from the same two-point jump.

Step 3. **So what.** Faster *and* more accurate, with the deepest context. The Transformer wins on every axis that matters for tagging.

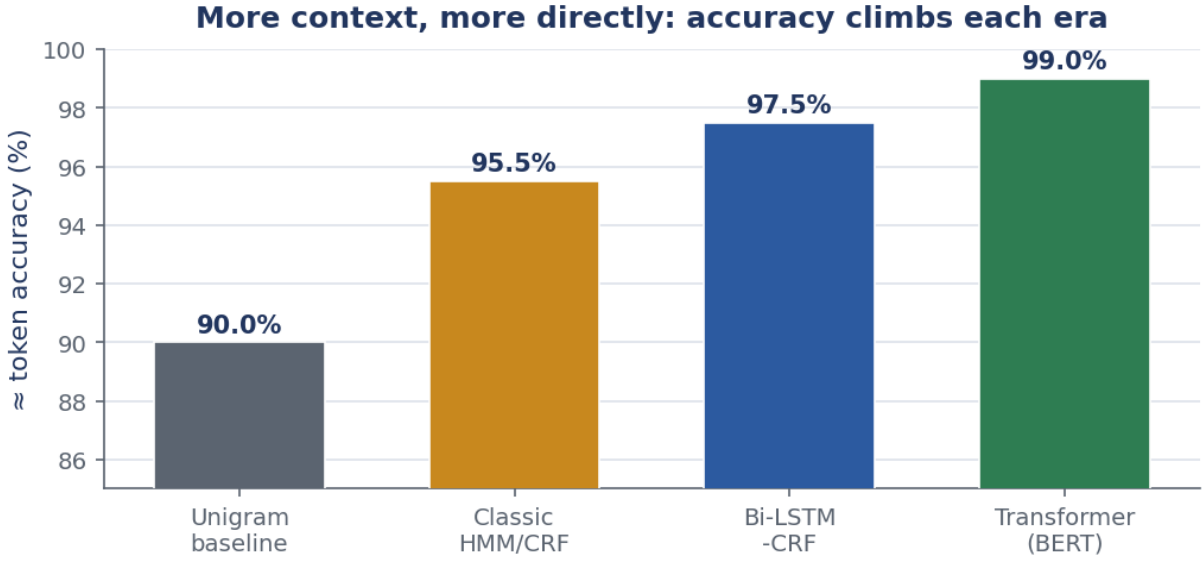

Figure 7: POS-tagging accuracy across eras, on Penn-Treebank-style benchmarks. From the unigram baseline, through classic HMM/CRF models, to the Bi-LSTM-CRF (about 97.5%) and fine-tuned Transformers (about 99%). Each method reads more context, more directly.

Impact. This is why POS tagging in modern libraries like **spaCy** and **Hugging Face Transformers** "just works." They wrap a fine-tuned Transformer. You call one function and get near-perfect tags.

✗ Watch out

"Parallel" means we process the positions of an input together. Generating text with an LLM is still left to right, one token at a time. And 99% is on clean benchmark text. Tweets, code, and noisy domains are harder.

Where this shows up in ML. The Transformer's parallelism is exactly what made training on internet-scale data practical. That, in turn, is what made today's large language models possible at all.

✓ Key takeaway

Transformers process a whole sentence in parallel and reach about 99% tagging accuracy. That is why they are the default in spaCy and Hugging Face.

10 Step 6 — Prompting: just *ask* the LLM

A large language model has read so much text that it can tag a sentence with no task-specific training at all. We just describe the job in words — a **prompt** — and read back the answer. No gradients, no fine-tuning corpus.

☞ The intuition

Fine-tuning rewires the model. Prompting just *instructs* it. The skill is already inside. The prompt is the question that calls it out. It is like asking a fluent speaker to label parts of speech, rather than sending them back to school.

There are two flavours. **Zero-shot** gives only the instruction. **Few-shot** adds a handful of solved examples first, so the model copies the format and the exact tag set.

✎ Worked example: tagging "The concert was great." by prompt

Zero-shot prompt (from the slide):

```
Tag the following sentence using the Penn Treebank tagset.
Output only the word/tag pairs.
Sentence:  The concert was great.
```

Step 1. **Expected output.** The/DT concert/NN was/VBD great/JJ ./. — determiner, noun, past verb, adjective, full stop.

Step 2. **A common zero-shot slip.** Left alone, the model may answer with the *wrong vocabulary*: great/ADJ or great/adjective instead of the Penn Treebank JJ. The grammar is right; the label convention is not.

Step 3. **Few-shot fix.** Add two solved examples in the exact tag set first:

```
Dogs/NNS bark/VBP ./.
She/PRP left/VBD ./.
Sentence:  The concert was great.
```

Now the model copies the convention and returns great/JJ.

Step 4. **So what.** A few in-context examples bought tag-set consistency with *zero* training. The prompt did the aligning that fine-tuning used to do.

✗ Watch out

LLMs can **hallucinate** a plausible-looking tag, drift off the requested tag set, or mis-split punctuation. For high-volume, accuracy-critical tagging, a fine-tuned Transformer is still cheaper and steadier. That sets up the agent idea next.

Where this shows up in ML. Zero- and few-shot prompting is the everyday interface to modern LLMs. The same pattern — describe the task, optionally show examples — drives translation, extraction, and classification.

✓ Key takeaway

A capable LLM can tag from a plain instruction (zero-shot). A few in-prompt examples (few-shot) lock it onto the right tag set, all with no fine-tuning.

11 Step 7 — LLM agents: tagging as a tool inside a bigger task

The final shift: POS tagging stops being the goal and becomes a *sub-step*. An **LLM agent** is a language model that can call **external tools** to get a job done. Given a messy request, it plans, calls a fast specialised tagger for the labelling part, and folds the result back into its reasoning.

☞ The intuition

The LLM is a smart project manager. It does not personally do every precise measurement. It hands the exact, repetitive part to a specialist tool, then writes up the result. Planning from the generalist, precision from the specialist.

★ Everyday picture

Asked "which rooms in this house face north?", you would not eyeball it. You would grab a compass (a tool), read each room, then answer. The agent grabs a POS tagger the same way, for the part that needs exactness.

The **agent loop** has five moves:

1. Read the request.
2. Break it into steps.
3. Call a tool for the step that needs it.
4. Read the tool's output.
5. Finish the task.

✎ Worked example: "Analyse the sentiment of all nouns in the user reviews"

Step 1. **Plan.** The agent splits the request. First, find the nouns. Second, judge the sentiment attached to each. Finding nouns needs accurate tagging — a job for a tool.

Step 2. **Call the tool.** POS_tag("The battery is amazing but the screen is dull .") returns tags. The agent keeps the NN words: **battery** and **screen**.

Step 3. **Use the tags.** For each noun it reads the describing word. "Battery" is paired

with "amazing" (+0.8). "Screen" is paired with "dull" (−0.6).

Step 4. Integrate and answer. It reports per-noun sentiment. Battery is positive (+0.8). Screen is negative (−0.6). That is exactly what the user asked for.

The LLM never had to be a great tagger itself. It *called* one (a fine-tuned BERT) and spent its own effort on planning and the write-up.

✗ Watch out

The agent must trust and correctly read the tool's output, and it can still mis-plan the overall task. Each tool call also costs time and money. The point is the division of labour, not magic. Precise sub-tasks go to precise tools.

Where this shows up in ML. This is **tool use** (or "function calling"). LLMs hand work to calculators, search, code runners, and taggers to stay accurate and current. POS tagging becomes one quiet utility among many inside larger AI systems.

✓ Key takeaway

An LLM agent uses a fast, specialised POS tagger as a tool. It combines the model's planning with the tagger's speed and accuracy, so tagging becomes a sub-step of a bigger task.

12 Conclusion: is POS tagging solved?

In a narrow sense, yes. Thanks to deep learning, POS tagging has reached near-human accuracy on standard benchmarks. The current state of the art is the fine-tuned Transformer (BERT, RoBERTa), at about 99%. And the frontier has moved. Tagging is now often an *internal utility* inside larger LLM agent systems, not a project in its own right.

☞ The intuition

The whole session is one trend in disguise. Each method uses *more context, more directly, more cheaply.* We climbed from a single word, to its neighbours, to every word at once, to all the language an LLM ever read. Finally we reached an LLM that simply calls a tagger when it needs one.

The seven steps, side by side:

Method	Context it uses	Training	≈ PTB accuracy
Unigram baseline	none (the word alone)	trivial lookup	≈ 90%
Classic HMM / MEMM / CRF	a few nearby tags	light	≈ 95–96%
Bi-LSTM-CRF	whole sentence, both directions	medium (slow, sequential)	≈ 97–98%
Transformer (BERT, RoBERTa)	whole sentence, in parallel	pre-train + fine-tune	≈ 98.5–99.5%
LLM prompting	everything it has read	none (just a prompt)	high, varies
LLM agent (POS as a tool)	LLM plan + the tagger's view	none (calls a tool)	the tool's accuracy

★ Everyday picture

It is the story of any maturing craft. First we obsess over building the perfect tool. Then the tool becomes reliable enough to disappear into the wall. We get on with what it was

for. POS tags now quietly power the next layer of systems.

Where this leaves you. The skill that matters now is less "build the best tagger" and more "use tags well." That means feeding clean syntactic structure into search, extraction, sentiment, and agent pipelines. The ladder you just climbed is the mental map for how almost every NLP task has modernised.

✓ Key takeaway

Modern POS tagging is effectively solved by fine-tuned Transformers. The live question has shifted from *how to build* the best tagger to *how to use* tags inside larger AI systems.

Further reading

A few of the sources cited on the lecture's reference slides, for going deeper:

- NLTK book, Ch. 5 — Categorizing and tagging words: `https://www.nltk.org/book/ch05.html`
- Stanford log-linear POS tagger: `https://nlp.stanford.edu/software/tagger.shtml`
- POS tagging using CRFs (tutorial): `https://towardsdatascience.com/pos-tagging-using-crfs-ea430c5fb78b`
- HMM, MEMM, and CRF — a comparative analysis: `https://www.alibabacloud.com/blog/hmm-memm-and-crf-a-comparative-analysis-of-statistical-modeling-methods_592049`
- POS tagging using RNNs: `https://towardsdatascience.com/pos-tagging-using-rnn-7f08a522f849`

Companion reader for Session 7 of Dr. Chetana Gavankar's Natural Language Processing course. Slides acknowledge Jurafsky & Martin and others. This companion expands every slide concept into plain English with fully worked numeric examples. The numbers in each worked example were computed and checked programmatically.